

# GridChunk 제작방식 정리 (방식 2 — 그리드 조합)

Assets/Scripts/Map/GridChunkDB/(런타임 데이터) + Assets/Scripts/Editor/MapGenerator/GridChunk/(에디터 툴)

기존 방식 1( AssembleChunk , 픽셀 커서 기반)과는 완전히 독립된 시스템이다. 겹침 판정을 픽셀 Rect 가 아니라 10 유닛 셀 격자 점유 여부( HashSet<Vector2Int> )로 하기 때문에, 청크를 만들 때 지켜야 하는 규칙이 방식 1과 다르다.

## 1. 청크 타입 (GridChunkType)

| 타입         | 의미                                                                                    | 크기              | 현재 사용 여부                |
|------------|---------------------------------------------------------------------------------------|-----------------|-------------------------|
| Content    | 전투/퍼즐/보물 등 세부 구분 없이 통합된 일반 청크. 실제 스폰되는 몬스터/함정/코인 등은 스폰 포인트 + SpawnKind 무작위 배정으로 정해진다. | 20×20<br>(2×2칸) | 사용 중                    |
| EndLine    | 진입점이 정확히 1개인 막다른길 전용. Start/End도 이 타입을 쓴다( GridEndLineRole 로 구분).                     | 20×20<br>(2×2칸) | 사용 중                    |
| Transition | 원래 통로 전용으로 두려 했던 타입                                                                   | 10×10 (1칸)      | 현재 생성기가 아예 선택하지 않음(미사용) |

Content와 EndLine을 반드시 같은 크기(20×20)로 맞추는 이유: 진입점 좌표 공식이 청크 자신의 width/height에 대한 상대값이라, 서로 연결되는 두 청크의 크기가 다르면 절대 좌표가 어긋나 정렬 가드에 걸린다. Transition은 지금 안 쓰이므로 크기 불일치 문제가 없다.

## 2. 진입점(EntranceSlot) 규칙

GridEntranceSlot 은 4방향 각 1개 슬롯짜리 [Flags] enum이다(원래 방향당 2슬롯이었다가 축소, 비트값은 하위호환을 위해 옛 값 유지).

| 슬롯    | 비트값 | 로컬 좌표 공식 (GridEntranceSlotResolver.Resolve) |
|-------|-----|---------------------------------------------|
| North | 1   | (width * 3 / 4, height)                     |
| South | 8   | (width * 3 / 4, 0)                          |
| East  | 16  | (width, height * 3 / 10)                    |
| West  | 64  | (0, height * 3 / 10)                        |

North/South는 같은 x 공식( width\*3/4 )을 써서 수직 통로가 대각선으로 어긋나지 않게 맞춰져 있다. 새 청크를 만들 때 이 공식을 벗어나는 별도 진입점을 추가하면 안 된다 — 이미 만든 다른 청크와 문이 안 맞물린다.

진입점 개수 = 청크 성격

생성기는 진입점 좌표가 아니라 **개수**로 체크 성격을 나눈다(별도 타입 필드 없음, `entrances.Count` 기준).

- **1개** → 막다른길( `IsDeadEndEntry` ). 메인/사이드 경로 진행 후보에서 자동 제외되고, `EndLine` 타입 체크만 이 역할을 맡는다. 폴더: `1Direction/`
- **2개** → 통로. 메인 경로·사이드 브랜치 진행에 쓰이는 일반 `Content` 체크. 폴더: `2Direction/`
- **3개 이상** → 분기( `IsThreeDirEntry` , `entrances.Count >= 3` ). 지나가면서 안 쓴 진입점이 사이드 브랜치 시작점으로 큐에 들어간다. 폴더: `3Direction/`

### 3. GridEndLineRole (EndLine 전용)

| 역할     | 값 | 용도                                  | 진입점 개수 |
|--------|---|-------------------------------------|--------|
| Start  | 1 | 맵 시작 지점. 원점(0,0)에 고정 배치             | 1개     |
| End    | 2 | 맵 목표 지점. 메인 경로 끝에 배치                | 1개     |
| Normal | 0 | 일반 막다른길. 메인 경로 도중 및 사이드 브랜치 마무리에 사용 | 1개     |

Start/End는 방향별로 1개씩(N/E/S/W) 만들어두면, 어느 방향으로 경로가 끝나든 백트래킹 없이 바로 붙는다 — 지금 `1Direction/`에 `End-E/N/S/W.prefab` 4개 + `Start.prefab` (East 고정) 1개가 이 형태로 있다.

### 4. 스폰 포인트 저작

`GridChunkData`에 `spawnPoints` (단일 `List<Vector2Int>`) 하나만 있고, 코인/몬스터/아이템/함정 등 **실제 종류는 저작하지 않는다** — 생성 시점에 `SpawnKind` 중 무작위로 정해진다(`GridSpawnResolver.ResolveAll`).

씬 뷰에서 좌표를 직접 입력하지 않고 클릭으로 찍을 수 있다(`GridChunkDataEditor`, 인스펙터의 "스폰 포인트 편집 모드" 토글):

- 빈 곳 클릭 → 추가
- 기존 점 Shift+클릭 → 삭제
- 기존 점 드래그 → 이동(정수 좌표 스냅)

Start/End 체크는 스폰 포인트 1개를 "Player 시작 위치" / "목표 위치" 마커로 쓴다 — 생성기가 그 좌표를 `GeneratedMapInfo.playerSpawnPosition` / `endPosition`에 기록만 하고, 실제 Player 생성·클리어 처리는 런타임 `GameManager`가 담당한다.

### 5. 폴더 구조 & 현재 제작 현황

```
Assets/Prefabs/Map/GridChunk/  
├── 1Direction/      Start.prefab, End-{E,N,S,W}.prefab, 1-{E,N,S,W}.prefab  (9개, EndLine)  
├── 2Direction/  
│   ├── E-N/  E-S/  E-W/  N-S/  W-N/  W-S/                                (방향쌍 x 3종 =  
├── 3Direction/      3Dir 1.prefab                                          (Content, entr  
└── GridChunkBase.prefab  (베이스/템플릿)
```

새 청크를 추가할 때는 진입점 개수에 맞는 폴더( 1Direction / 2Direction / 3Direction )에 넣는 것을 관례로 따른다(스캔 자체는 하위 폴더 전체를 재귀 검색하므로 폴더명이 동작에 영향을 주진 않는다 — 사람이 보기 위한 정리용 규칙).

## 6. 새 청크 제작 체크리스트

---

1. `GridChunkBase.prefab` 을 복제하거나, `GridChunkData` 컴포넌트를 붙인 새 프리팹을 만든다.
2. `chunkType` 지정 — 막다른길이면 `EndLine (+ endLineRole)`, 통로/분기면 `Content`.
3. `entranceSlots` 에 필요한 방향만 체크. 진입점 개수가 곧 청크 성격(1=막다른길, 2=통로, 3+=분기)이 되므로 의도한 개수와 일치하는지 확인.
4. 타일 아트를 \$2의 좌표 공식( $\text{North/South} = \text{width} * 3 / 4$ ,  $\text{East/West} = \text{height} * 3 / 10$ )에 맞춰 배치 — 어긋나면 다른 청크와 안 이어진다.
5. 필요하면 스폰 포인트를 씬 뷰 클릭 툴로 찍는다.
6. `Assets/Prefabs/Map/GridChunk/` 하위, 개수에 맞는 폴더에 저장.
7. **Map > Rebuild Grid Chunk Database** 실행 — `GridChunkDatabaseBuilder` 가 `Assets/Prefabs/Map/GridChunk` 전체를 스캔해서 `GridChunkDatabase.asset` 을 재생성한다 (`GridChunkData` 없는 프리팹은 자동 스킵).

## 7. 생성기 실행 — Map > Generate Grid Map

---

`GridChunkMapGeneratorWindow` → `ChunkGridGenerator.GenerateGrid(db, contentChunkCount, seed, spawnPrefabs, maxThreeDirCount)`

- **Content Chunk Count:** 메인 경로 Content 개수를 정확히 이 값으로 맞춘다. End가 안 붙으면 늘리지 않고 마지막 청크(들)를 다른 후보로 교체하며 백트래킹한다.
- **Max 3Direction Count:** 분기(3개 이상 진입점) 청크를 메인+사이드 합쳐 총 몇 개까지 허용할지. 한도 도달 시 후보에서 사전 제외(사후 삭제 없음 — 이미 연결된 청크를 지우면 다른 경로가 끊어지는 위험이 있어서).
- **사이드 브랜치:** 분기 청크의 안 쓴 진입점마다 무작위 2~5개 목표로 진행하다가(불가능하면 0~1개도 허용), 1Direction ( `EndLineRole.Normal` ) 청크로 마무리. 자리가 없으면 조용히 skip.
- 생성 완료 후 `GeneratedMapInfo` 컴포넌트(루트)에 `playerSpawnPosition / endPosition` 만 기록 — Player 생성/클리어 처리는 하지 않는다.

## 8. 핵심 불변식 (요약)

---

- 서로 연결되는 청크는 반드시 같은 **width/height**를 가져야 한다(그래야 진입점 공식 상대값이 절대 좌표로도 일치).
- 청크 원점은 항상 `CellSize` (10)의 배수여야 한다 — 어긋나면 정렬 가드( `origin % 10 != 0` )에 걸려 그 후보는 자동 제외된다.
- 진입점 개수가 청크의 역할(막다른길/통로/분기)을 결정한다 — 별도 타입 필드로 구분하지 않는다.